

Manuale: Deploy WellBeingApp API su Railway

Obiettivo

Deployare WellBeingApp API (.NET 9) su Railway con PostgreSQL managed, zero-config SSL, auto-deploy da Git.

1. Perché Railway per WellBeingApi

Pro | Contro

Deploy da Git push (zero DevOps) | Costo più alto di una VPS pura (~5-20 USD/mese)
PostgreSQL managed incluso | Nessun accesso SSH al container
SSL automatico + dominio custom | Cold start possibile su Hobby plan
Scaling orizzontale facile | Egress limitato su free tier
Logs, metriche, health check integrati | Vendor lock-in leggero (ma è Docker, portabile)
Nixpacks auto-detect .NET | Storage effimero (serve volume o external storage)

Ideale per: progetto personale/MVP che non vuoi gestire infrastrutturalmente, con budget ~5-15 USD/mese.

2. Prerequisiti

- Account Railway (GitHub login consigliato)
- Repository Git con il codice WellBeingApp (GitHub o GitLab)
- API key DashScope (Qwen text + TTS)
- Google OAuth Client ID (per login social)
- Facebook App ID + Secret (opzionale)

3. Struttura Progetto Railway

```
Railway Project: "WellBeingApp"  
├── Service: wellbeing-api (.NET 9)  
├── Service: PostgreSQL (plugin managed)  
└── (opzionale) Service: Redis (se serve caching)
```

4. Setup Passo-Passo

4.1 Crea il progetto

- Railway Dashboard → New Project
- Scegli "Deploy from GitHub repo"
- Autorizza Railway ad accedere al tuo repository
- Seleziona il repo WellBeingApp

4.2 Configura il servizio API

Railway auto-detecta .NET via Nixpacks. Configura:

Campo | Valore

Root Directory | /WellBeingApi

Build Command | (auto: dotnet publish -c Release -o out)

Start Command | dotnet WellBeingApi.dll

Port | 8080

Se Nixpacks non rileva correttamente, aggiungi un Dockerfile nella root del repo (vedi sezione 4.6).

4.3 Aggiungi PostgreSQL

- Nel progetto Railway → + New → Database → PostgreSQL
- Railway crea l'istanza e inietta automaticamente la variabile DATABASE_URL
- Il formato è: postgresql://user:password@host:port/dbname

4.4 Configura Variabili Ambiente

Railway Dashboard → Service "wellbeing-api" → Variables:

```
# === .NET Runtime ===
ASPNETCORE_ENVIRONMENT=Production
ASPNETCORE_URLS=http://+:8080
PORT=8080

# === Database ===
# Railway inietta DATABASE_URL automaticamente dal plugin PostgreSQL.
# Il tuo codice usa ConnectionStrings__Postgres, quindi mappa:
ConnectionStrings__Postgres=${{Postgres.DATABASE_URL}}
UserDataStorage__Provider=Postgres

# === JWT ===
Jwt__Key=<GENERA-CHIAVE-ALFANUMERICA-MINIMO-32-CARATTERI>
Jwt__Issuer=https://wellbeing-api.up.railway.app
Jwt__Audience=WellBeingApp
Jwt__ExpireMinutes=1440

# === AI Services (Qwen/DashScope) ===
QwenDashScopeTextGenerator__ApiKey=<TUA-DASHSCOPE-API-KEY>
QwenDashScopeTextGenerator__Model=qwen3.6-flash
QwenDashScopeTextGenerator__Endpoint=https://dashscope-intl.aliyuncs.com/compatible-mode/v1
Qwen3Ts__ApiKey=<TUA-DASHSCOPE-API-KEY>

# === Social Login ===
Google__ClientId=<TUO-GOOGLE-OAUTH-CLIENT-ID>
Facebook__AppId=<TUO-FACEBOOK-APP-ID>
Facebook__AppSecret=<TUO-FACEBOOK-APP-SECRET>

# === Admin Panel ===
Admin__Username=admin
Admin__Password=<GENERA-PASSWORD-SICURA>

# === Opzionale: Azure AI Agents ===
# AzureAiAgent__ProjectEndpoint=<endpoint>
# AzureAiAgent__AgentId=<id>
```

****Nota sulla variabile DATABASE_URL:**** Railway la fornisce nel formato standard PostgreSQL. Se il tuo `Program.cs` legge da `ConnectionStrings:Postgres`, devi mapparla. Usa la sintassi `\${{Postgres.DATABASE\_URL}}`.

di Railway per referenziare il servizio database, oppure copia il valore manualmente.

4.5 Mapping variabile DATABASE_URL → formato .NET

Se il codice usa IConfiguration con ConnectionStrings:Postgres nel formato Npgsql (Host=...;Port=...;Database=...), aggiungi questo in Program.cs per supportare entrambi i formati:

```
// Program.cs — aggiungi prima di builder.Build()
var databaseUrl = Environment.GetEnvironmentVariable("DATABASE_URL");
if (!string.IsNullOrEmpty(databaseUrl))
{
    var uri = new Uri(databaseUrl);
    var userInfo = uri.UserInfo.Split(':');
    var connStr = $"Host={uri.Host};Port={uri.Port};Database={uri.AbsolutePath.TrimStart('/')};Username={userInfo[0]};Pa
builder.Configuration["ConnectionStrings:Postgres"] = connStr;
}
```

Oppure, se preferisci non toccare il codice, componi manualmente la stringa in Railway:

```
ConnectionStrings__Postgres=Host=${{Postgres.PGHOST}};Port=${{Postgres.PGPORT}};Database=${{Postgres.PGDATABASE}};Userna
```

4.6 Dockerfile (se Nixpacks non funziona)

Se Railway non builda correttamente con Nixpacks, crea un Dockerfile nella root del repo:

```

# Dockerfile (root del repo WellBeingApp)
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src

# Restore
COPY WellBeingApp.sln .
COPY WellBeingApi/*.csproj WellBeingApi/
COPY WellBeing/*.csproj WellBeing/
COPY SharedStuff/*.csproj SharedStuff/
COPY WellBeingApiFunction/*.csproj WellBeingApiFunction/
COPY WellBeingFunctions/*.csproj WellBeingFunctions/
COPY WellBeingApi.Tests/*.csproj WellBeingApi.Tests/
RUN dotnet restore WellBeingApi/WellBeingApi.csproj

# Build
COPY . .
RUN dotnet publish WellBeingApi/WellBeingApi.csproj \
  -c Release \
  -o /app/publish \
  --no-restore

# Runtime
FROM mcr.microsoft.com/dotnet/aspnet:9.0
WORKDIR /app
RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*
COPY --from=build /app/publish .

ENV ASPNETCORE_URLS=http://+:8080
ENV PORT=8080
EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=10s --start-period=45s --retries=3 \
  CMD curl -f http://localhost:8080/health/live || exit 1

ENTRYPOINT ["dotnet", "WellBeingApi.dll"]

```

Poi in Railway Settings:

- Builder: Dockerfile
- Dockerfile Path: ./Dockerfile

4.7 Configura railway.toml (opzionale)

Crea railway.toml nella root del repo per configurazione declarativa:

```

[build]
builder = "dockerfile"
dockerfilePath = "./Dockerfile"

[deploy]
startCommand = "dotnet WellBeingApi.dll"
healthcheckPath = "/health/live"
healthcheckTimeout = 45
restartPolicyType = "on_failure"
restartPolicyMaxRetries = 3

[service]
internalPort = 8080

```

4.8 Dominio Custom

- Railway Dashboard → Service → Settings → Networking → Custom Domain

- Aggiungi `wellbeing-api.tuodominio.it`
- Railway ti dà un CNAME target (es. `wellbeing-api-production-xxxx.up.railway.app`)
- Nel tuo DNS provider, crea:

```
CNAME wellbeing-api → <cname-fornito-da-railway>
```

- Railway genera il certificato SSL automaticamente

Aggiorna la variabile:

```
Jwt__Issuer=https://wellbeing-api.tuodominio.it
```

5. Deploy e CI/CD

5.1 Auto-deploy da Git

Railway deploys automaticamente ad ogni push sul branch configurato (default: main).

Evento | Azione Railway

Push su main | Build + Deploy automatico

Pull Request | Deploy preview (ambiente isolato)

Rollback | Un click dalla UI (deploy precedente)

5.2 Railway CLI (opzionale)

```
# Installa CLI
npm install -g @railway/cli

# Login
railway login

# Link progetto locale
cd c:\CodeM\Personal\WellBeingApp
railway link

# Deploy manuale
railway up

# Logs in tempo reale
railway logs

# Apri shell nel container (per debug)
railway shell

# Variabili ambiente locali (per sviluppo)
railway run dotnet run --project WellBeingApi
```

6. Database: Migrations e Seed

6.1 Esegui migrations al primo deploy

Se usi EF Core migrations, il modo più semplice è eseguirle all'avvio dell'app. Verifica che `Program.cs` contenga:

```
// Applica migrations automaticamente in produzione
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<YourDbContext>();
    await db.Database.MigrateAsync();
}
```

6.2 Migrations manuali via CLI

```
# Connettiti al DB Railway dalla tua macchina
railway run dotnet ef database update --project WellBeingApi
```

Oppure usa il connection string dal Railway Dashboard (Variables → Postgres → DATABASE_URL) con dotnet ef:

```
dotnet ef database update \
    --project WellBeingApi \
    --connection "Host=<host>;Port=<port>;Database=<db>;Username=<user>;Password=<pw>;SSL Mode=Require"
```

7. Stima Costi Railway

7.1 Pricing Model

Railway usa un modello pay-per-use basato su:

- vCPU: \$0.000463/min (~\$20/mese per 1 vCPU continuo)
- RAM: \$0.000231/GB/min (~\$10/mese per 1 GB continuo)
- Disco: \$0.000257/GB/min
- Egress: \$0.10/GB dopo i primi 100 GB

7.2 Stima per WellBeingApp

Risorsa | Consumo stimato | Costo/mese

API Service (~0.25 vCPU, ~300 MB RAM) | Always-on | ~\$5-7

PostgreSQL (~256 MB RAM, 1 GB disco) | Always-on | ~\$3-5

Egress (API calls, ~5 GB) | Variabile | ~\$0.50

TOTALE stimato | ~\$8-12/mese

7.3 Piano Hobby vs Pro

Feature | Hobby (\$5 credit incluso) | Pro (\$20 minimo)

Trial credit | \$5/mese inclusi | Nessun cap fisso

Sleep after inactivity | Si (dopo 10 min) | No (always-on)

Custom domains | Si | Si

Team members | 1 | Illimitati

Preview deployments | Si | Si

Private networking | Si | Si

****Per un'app mobile con utenti reali:**** usa il piano ****Pro**** (~\$20/mese minimo) per evitare cold start che degradano l'UX. Il piano Hobby va bene per dev/staging.

8. Monitoring e Observability

8.1 Railway Built-in

- Logs: streaming real-time dalla dashboard
- Metrics: CPU, RAM, Network in/out per servizio
- Deploy history: ogni deploy con status, durata, commit

8.2 Health Checks

Railway monitora automaticamente il tuo healthcheckPath. Se fallisce:

- 3 retry in 45s
- Se persiste → rollback automatico al deploy precedente

8.3 Alerting (opzionale)

Railway non ha alerting nativo avanzato. Opzioni:

- Uptime Kuma (self-hosted sulla VPS Coolify, se ce l'hai)
- BetterStack (free tier: 5 monitors)
- Healthchecks.io (free tier: 20 checks)

9. Sicurezza

9.1 Best Practices Railway

- [] Tutte le variabili sensibili in Railway Variables (mai nel codice)
- [] JWT Key di almeno 32 caratteri alfanumerici random
- [] SSL forzato su PostgreSQL (SSL Mode=Require)
- [] CORS configurato solo per i domini della tua app mobile
- [] Rate limiting attivo sugli endpoint API (già presente nel codice)
- [] Admin panel protetto (cookie auth + credenziali forti)

9.2 Rotazione Secrets

Per ruotare una chiave:

- Railway Dashboard → Variables → modifica il valore
- Railway ri-deploya automaticamente il servizio
- Se ruoti Jwt__Key: tutti i token attivi diventano invalidi (gli utenti dovranno ri-loggarsi)

10. Workflow di Sviluppo

10.1 Ambienti

GitHub Branch	Railway Environment
main	→ Production (wellbeing-api.tuodominio.it)
develop	→ Staging (auto-generated URL)
feature/*	→ Preview (temporaneo, per PR)

Configura in Railway: Settings → Environments → aggiungi "staging" linked a branch develop.

10.2 Flusso tipico

1. Sviluppo locale (dotnet run + PostgreSQL locale/Docker)
2. Push su feature branch → Railway crea preview environment
3. Test su preview URL
4. Merge in main → Deploy automatico in produzione

10.3 Sviluppo locale con variabili Railway

```
# Scarica le variabili di produzione in locale (senza copiarle a mano)
railway run -- dotnet run --project WellBeingApi
```

```
# Oppure esporta in .env locale
railway variables --format dotenv > .env.railway
```

11. Checklist Deployment

Prima volta

- ☐ Account Railway creato (piano Hobby o Pro)
- ☐ Repository GitHub collegato
- ☐ Dockerfile o Nixpacks funzionante
- ☐ PostgreSQL plugin aggiunto al progetto
- ☐ Tutte le variabili ambiente configurate
- ☐ Health check risponde: GET /health/live → 200
- ☐ Migrations eseguite (database popolato)
- ☐ Login Google funzionante (test dall'app mobile)
- ☐ Endpoint AI risponde: POST /api/ai/counselor/text → 200
- ☐ Dominio custom configurato + SSL attivo
- ☐ Jwt__Issuer aggiornato al dominio finale

Ad ogni release

- ☐ Push su main → verifica deploy success in Railway dashboard
- ☐ Controlla logs per errori post-deploy
- ☐ Verifica health check verde
- ☐ Test rapido dall'app mobile (login + una richiesta AI)

12. Troubleshooting

Problema	Causa	Soluzione
----------	-------	-----------

Build fallisce: "project not found" | Root directory errata | Imposta /WellBeingApi o usa Dockerfile
SSL connection required | PostgreSQL Railway richiede SSL | Aggiungi SSL Mode=Require;Trust Server Certificate=true alla connection string
502 dopo deploy | App non in ascolto sulla porta giusta | Verifica PORT=8080 e ASPNETCORE_URLS=http://+:8080
Cold start lento (5-10s) | Piano Hobby, container si spegne dopo inattività | Passa a Pro, oppure aggiungi keep-alive ping
JWT validation failed | Key troppo corta o Issuer mismatch | Minimo 32 chars; Issuer = URL dominio con https://
AI endpoint 429 | Rate limit interno (10 req/min) | E il comportamento atteso; il client deve gestire il retry
EF Migration fallisce | Schema incompatibile | railway run dotnet ef migrations add Fix poi redeploy
Out of memory | Container troppo piccolo | Railway scala automaticamente, ma verifica il consumo in Metrics

13. Confronto: Railway vs VPS+Coolify per WellBeingApi

Aspetto	Railway	VPS + Coolify
Setup time	15 minuti	1-2 ore
Costo/mese	\$8-15 (variabile)	€7-14 (fisso)
Maintenance	Zero	Aggiornamenti OS, backup manuali
Scaling	Automatico	Manuale (resize VPS)
Cold start	Si (Hobby) / No (Pro)	No (always-on)
Backup DB	Automatico (Point-in-time)	Da configurare
Custom domain	Si	Si
Monitoring	Built-in base	Da aggiungere (Uptime Kuma)
Vendor lock-in	Basso (e Docker)	Nessuno
Ideale per	MVP, side project, bassa ops	Controllo totale, multi-servizio

****Raccomandazione:**** Usa Railway per WellBeingApi (side project, pochi utenti, zero ops). Usa la VPS+Coolify per AIA Platform (multi-servizio complesso, costi predicibili, controllo totale).

Ultimo aggiornamento: 2026-06-12